

What this project is, in plain English

Imagine you're a doctor — but instead of a patient, you're examining a software codebase. Most checkup tools only tell you the patient's current weight. This project gives you the full medical chart — every change since birth, with vital signs over time, so you can see whether the patient is getting healthier or sicker.

The everyday analogy

Think of a codebase like a kitchen used by a restaurant. Every day cooks come in and rearrange things — add a new burner here, store the salt over there, hire a new chef, remove an old recipe. After two years, the kitchen still *works*, but no one really knows:

- **Is it harder to cook in than it was a year ago?** (Has complexity quietly grown?)
- **Where do most accidents happen?** (Which files cause the most bugs?)
- **If chef Tarin quits tomorrow, what stops working?** (Who knows which corner of the kitchen?)
- **If we accept this new recipe, will the kitchen get messier?** (Will merging this PR hurt us?)

Repo Health Intelligence answers all four questions automatically, with numbers, charts, and explanations.

The one-screen view

Every codebase gets a single Health Score from 0 to 100

It's not a black box. The score is made of five clearly named ingredients (we call them subscores). If the score drops, you can see exactly which ingredient went bad, click the dip, and read a plain-language explanation of why.

The five ingredients of the Health Score

Complexity drift	Is the code getting more tangled over time?	0 = same as day 1 · 1 = much worse
Test coverage	How much of the code has safety nets (tests) on it?	0 = no tests · 1 = lots of tests
Hotspot risk	Are bugs concentrating in a few risky files?	0 = evenly spread · 1 = one file is a bomb

Dependency rot	How many outside libraries do we lean on, and how long since we touched them?	0 = healthy · 1 = stale and bloated
Architectural drift	Are the parts of the system getting tangled (circular dependencies, orphans)?	0 = clean structure · 1 = spaghetti

What it found in a real project

We pointed the system at a public learning-platform codebase called Edulume. It read all 109 commits (every change ever made to the project), built a knowledge map of every file, and computed the five subscores at every point in history. The whole thing took about 30 seconds on a laptop.

The headline numbers

55.2 Starting health	28.2 Today's health	-27.0 Net drop	~270 Days alive
--------------------------------	-------------------------------	--------------------------	---------------------------

Translation: the project started in decent shape but has been quietly getting worse for almost a year. Without this tool, nobody on the team would notice.

Specific things the system caught

- **Biggest single drop happened the day someone added a discussion forum** — health fell 18 points overnight because the new feature shipped without tests and doubled the size of the codebase.
- **One file (`client/src/utils/api.ts`) is the #1 risk** — touched 19 times, very complex. If something breaks, it's almost certainly here.
- **Two files actually depend on each other in a circle** — `TestTimeRemaining` ↔ `TestPageStandalone`. This is a textbook bug magnet.
- **One person (the founder) owns 87% of the server code and 100% of the auth code.** If they go on vacation, no one else knows how the login flow works.
- **If you were to merge the experimental "olive theme" branch**, the system predicts health would drop another 2.1 points — mostly because it concentrates more bug risk into already-risky files.

Why this matters

These five findings are exactly the kind of insight an experienced engineer would surface in a code review — but only if they spent days reading every commit. The system does it in 30 seconds and updates as the team ships.

Everything the brief asked for, plus four bonus features

The brief asked for five things. All five are done.

- ✓ **Walk a repo's history and build a per-commit code map.** Tree-sitter parser handles 5 programming languages.
- ✓ **A health score with at least 4 explainable subscores.** We built 5. Each one has a one-line explanation.
- ✓ **Time-series dashboard with PR-level annotations.** Health curve, with red dots marking moments the score dropped.
- ✓ **Knowledge graph diff — compare any two commits.** Shows files added, removed, restructured, plus changed imports.
- ✓ **Hotspot map — where do bugs live.** Ranked list with bar visualization, color-graded.

The brief also listed bonus objectives. We hit four of them.

- ★ **Bus factor per module.** Shows who owns each part of the code and which parts collapse if one person leaves.
- ★ **Architectural drift detection.** Catches when files start forming dependency circles or become disconnected.
- ★ **LLM 'why did health drop' narrative.** Click any dip → AI writes a 2-3 sentence explanation, with a free fallback when no API key is set.
- ★ **Pre-merge health prediction.** Pick any open branch, see exactly how merging it would shift the health score before you ship.
- ★ **Multi-language support.** TypeScript, JavaScript, JSX/TSX, and Python all supported.

Bonus extras we added on our own

- **Interactive knowledge-graph visualization** — every file as a glowing node, every import as a line, complexity shown by node size and color.
- **Circular-dependency detection** — finds tangled code automatically using a classic graph algorithm (Tarjan SCC).
- **Orphan-file detector** — flags files that nothing imports and that import nothing — often forgotten dead code.
- **Tabbed dashboard UI** with a distinct violet/cyan visual identity and animated brand mark.

Why our AI usage is cheap, auditable, and optional

The hackathon brief specifically said "LLM usage must be cost-justified — every call defended." Here's our discipline:

- **Only one part of the app calls an AI model** — the "explain this dip" button.
- **It only runs when a human clicks it** — never automatically, never during ingestion. Reading a repo costs zero AI calls.
- **We send small, structured prompts** — not source code. The prompt is a short list of numbers (the score deltas) plus a few file names. About 600 tokens regardless of repo size.
- **Every answer is cached forever.** Clicking the same dip twice doesn't re-run the model.
- **Output is capped at 280 tokens.** A single dip explanation costs a fraction of a US cent on Claude Haiku.
- **Without an API key, the feature still works** — we fall back to a deterministic rule-based summary written from the same data. The user never sees a broken button.

What it's built with

The brain (offline)

Node.js script reads git history, parses every file with tree-sitter (an industry-standard code parser used by GitHub itself), and stores the results in a local SQLite database. No cloud needed.

The face (web app)

Next.js + React dashboard. Charts via Recharts. Knowledge graph drawn on HTML canvas with a custom force-directed layout. Optional Claude Haiku narratives via Vercel AI Gateway.

Where to find it

Code: github.com/avinrique/repo-health-intelligence

Demo repo: github.com/tarinagarwal/Edulume (109 commits, real-world)

Pitch deck: [repo-health-intelligence.pptx](#) in the same directory as this PDF

If you have 60 seconds, look at this

The single most powerful demo

On the dashboard, click the biggest red dot on the health chart. The system instantly tells you it was the "add discussion forum" commit, that health dropped 18 points, that the cause was complexity growth and missing tests, and that two specific files (`DiscussionDetailPage.tsx`, `server/routes/discussions.js`) carry most of the new risk — all without you ever reading the code.

That's the whole pitch in one click: any team, on any repo, gets a doctor's-eye view of their codebase's vital signs over time.